

Understanding SQL

In this chapter, you'll learn about databases and SQL, which are prerequisites to learning MySQL.

Database Basics

The fact that you are reading this book indicates that you, somehow, need to interact with databases, and MySQL specifically. And so, before diving into MySQL and its implementation of the SQL language, it is important that you understand some basic concepts about databases and database technologies.

Whether you are aware of it or not, you use databases all the time. Each time you select a name from your email address book, you are using a database. When you browse contacts on your phone, you are using a database. If you conduct a search on an Internet search site, you are using a database. When you log in to your network at work, you are validating your name and password against a database. Even when you use your ATM card at a cash machine, you are using databases for PIN verification and balance checking.

But even though we all use databases all the time, there remains much confusion over what exactly a database is. This is especially true because different people use the same database terms to mean different things. Therefore, a good place to start our study is with a list and explanation of the most important database terms.

Tip

Reviewing Basic Concepts What follows is a very brief overview of some basic database concepts. It is intended to either jolt your memory if you already have some database experience or to provide you with the absolute basics if you are new to databases. Understanding databases is an important part of mastering MySQL, and you might want to find a good book on database fundamentals to brush up on the subject, if needed.

What Is a Database?

The term *database* is used in many different ways, but for our purposes in this book, a database is a collection of data stored in some organized fashion. The simplest way to think of it is to imagine a database as a filing cabinet. The filing cabinet is simply a physical location to store data, regardless of what that data is or how it is organized.

New Term

Database A container (usually a file or set of files) for storing organized data.

Caution

Misuse Causes Confusion People often use the term *database* to refer to the database software they are running. This is incorrect, and it is a source of much confusion. Database software is actually called a *database management system* (or *DBMS*). A database is a container created and manipulated via a DBMS. A database might or might not be a file stored on a hard drive. And for the most part, this is not even significant as you never access a database directly anyway; you always use the DBMS, and it accesses the database for you.

Tables

When you store information in a filing cabinet, you don't just toss it in a drawer. Rather, you create files within the filing cabinet, and then you store related data in specific files.

In the database world, a file is called a *table*. A table is a structured file that can store data of a specific type. A table might contain a list of customers, a product catalog, or any other list of information.

New Term

Table A structured list of data of a specific type.

The key here is that the data stored in the table is one type of data or one list. You would never store a list of customers and a list of orders in the same database table. Doing so would make subsequent retrieval and access difficult. Rather, you'd create two tables, one for each list.

Every table in a database has a name that identifies it. That name is always unique—meaning no other table in that database can have the same name.

Note

Table Names What makes a table name unique is actually a combination of several things, including the database name and table name. While you cannot use the same table name twice in the same database, you definitely can reuse table names in different databases.

Tables have characteristics and properties that define how data is stored in them. These include information about what data may be stored, how it is broken up, how individual pieces of information are named, and much more. The set of information that describes a table is known as a *schema*, and a schema can be used to describe specific tables within a database, as well as an entire database (and the relationship between tables in a database, if any).

New Term

Schema Information about database and table layout and properties.

Note

Schema or Database? Occasionally the term *schema* is used as a synonym for *database* (and *schemata* as a synonym for *databases*). While unfortunate and frequently confusing, it is usually clear from the context which meaning of schema is intended. In this book, *schema* is used as defined here.

Columns and Datatypes

Tables are made up of columns. A column contains a particular piece of information within a table.

New Term

Column A single field in a table. Every table is made up of one or more columns.

The best way to understand this is to envision database tables as grids, somewhat like spreadsheets. Each column in the grid contains a particular piece of information. In a customer table, for example, the customer number is stored in one column, the customer name is stored in another, and the address, city, state, and zip code are all stored in their own columns.

Tip

Breaking Up Data It is extremely important to break data into multiple columns correctly. For example, city, state, and zip code should always be stored in separate columns. By breaking these out, it becomes possible to sort or filter data by specific columns (for example, to find all customers in a particular state or in a particular city). If city and state are combined into one column, it would be extremely difficult to sort or filter by state.

Each column in a database has an associated datatype. A datatype defines what type of data the column can contain. For example, if a column is to contain a number (perhaps the number of items in an order), it would be associated with the numeric datatype.

Columns that contain dates, text, notes, currency amounts, and so on would use the appropriate datatypes.

New Term

Datatype A type of allowed data. Every table column has an associated datatype that restricts (or allows) specific data in that column.

Datatypes restrict the type of data that can be stored in a column (for example, preventing the entry of alphabetical characters into a numeric field). Datatypes also help sort data correctly and play an important role in optimizing disk usage. As such, special attention must be given to picking the right datatype when tables are created.

Rows

Data in a table is stored in rows; each record saved is stored in its own row. Again, if you envision a table as a spreadsheet-style grid, the vertical columns in the grid are the table columns, and the horizontal rows are the table rows.

For example, a customers table might store one customer per row. The number of rows in the table is the number of records in the table.

New Term

Row A record in a table.

Note

Records or Rows? You might hear users refer to database *records* when referring to *rows*. For the most part, the two terms are used interchangeably, but *row* is technically the correct term.

Primary Keys

Every row in a table should have some column (or set of columns) that uniquely identifies it. A table containing customers might use a customer number column for this purpose, whereas a table containing orders might use the order ID. Similarly, an employee list table might use an employee ID column.

New Term

Primary Key A column (or set of columns) whose values uniquely identify every row in a table.

The column (or set of columns) that uniquely identifies each row in a table is called a *primary key*. The primary key is used to refer to a specific row. Without a primary key, updating or deleting specific rows in a table is extremely difficult because there is no guaranteed safe way to refer to just the rows that are affected.

Tip

Always Define Primary Keys Although primary keys are not actually required, most database designers ensure that every table they create has a primary key so that future data manipulation is possible and manageable.

Any column in a table can be established as the primary key, as long as it meets the following conditions:

- No two rows can have the same primary key value.
- Every row must have a primary key value. (Primary key columns may not allow NULL values.)

Note

Primary Key Rules The rules listed here are enforced by MySQL itself.

A primary key is usually defined on a single column within a table. But this is not required, and multiple columns may be used together as a primary key. When multiple columns are used, the rules previously listed must apply to all columns that make up the primary key, and the values of all columns together must be unique. (Individual columns need not have unique values.)

Tip

Primary Key Best Practices In addition to following the rules that MySQL enforces, you should adhere to several universally accepted best practices:

- Don't update values in primary key columns.
- Don't reuse values in primary key columns.
- Don't use values that might change in primary key columns. (For example, if you use a name as a primary key to identify a supplier and the supplier merges with another company and changes its name, you have to change the primary key.)

There is another very important type of key, called a *foreign key*, but I'll get to that in Chapter 15, "Joining Tables."

What Is SQL?

SQL (pronounced as the letters “S-Q-L” or as the word “sequel”) is an abbreviation for Structured Query Language. SQL is a language designed specifically for communicating with databases.

Unlike other languages (spoken languages such as English or programming languages such as Python or Java), SQL is made up of very few words. This is deliberate. SQL is designed to do one thing and do it well: provide you with a simple and efficient way to read and write data from a database.

What are the advantages of SQL?

- SQL is not a proprietary language used by specific database vendors. Almost every major DBMS supports SQL, and learning this one language enables you to interact with just about every database you’ll run into.
- SQL is easy to learn. The statements are all made up of descriptive English words, and there aren’t that many of them.
- Despite its apparent simplicity, SQL is actually a very powerful language, and by cleverly using its language elements, you can perform very complex and sophisticated database operations.

Note

DBMS-Specific SQL Although SQL is not a proprietary language and there is a standards committee that tries to define SQL syntax that can be used by all DBMSs, the reality is that no two DBMSs implement SQL identically. The SQL taught in this book is specific to MySQL, and while much of the language taught will be usable with other DBMSs, you should not assume complete SQL syntax portability.

Try It Yourself

All of the chapters in this book use working examples, showing you the SQL syntax, what it does, and explaining why it does it. I strongly suggest that you try each and every example for yourself so that you learn MySQL firsthand.

In addition, starting in Chapter 4, “Retrieving Data,” most chapters conclude with a “Challenges” section to help you review and gauge your MySQL proficiency. If you get stuck, you can go to the companion website to find the answers to the “Challenges” section questions.

Appendix B, “The Example Tables,” describes the example tables used throughout this book and explains how to obtain and install them. If you have not done so yet, take a look at that appendix before proceeding.

Note

You Need MySQL Obviously, you'll need access to a copy of MySQL to follow along. Appendix A, "Getting Started with MySQL," explains where to get a copy of MySQL and provides some pointers for getting started. If you do not have access to a copy of MySQL, read that appendix before proceeding.

Summary

In this first chapter, you learned what SQL is and why it is useful. Because SQL is used to interact with databases, you also reviewed some basic database terminology.